QUANTUM HACKATHON | WEEK ONE PROBLEMS

Attempt due on Canvas by Wednesday, February 25 at 23:59
(to be completed as a team and submitted by your team lead)

Team Name:

Team Lead:

Other Team Members:

Problem 1

This problem concerns the following matrix (written in the computational basis), which you'll recall is called the *Hadamard gate*:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

In quantum computing, H is one of the most fundamental quantum gates, as it appears in almost every interesting quantum algorithm we know.

(a) Prove that $H^\dagger = H$. Conclude that H is *Hermitian*.

(b) Prove that $H \in \mathrm{U}(2)$. Conclude that H is an *involution*.

(c) Let $H^{\otimes n}$ denote the n-fold tensor product $H \otimes H \otimes \cdots \otimes H$ and let 0^n denote the n-bit, all-zero string $00\ldots0$. Prove that the probability distribution one obtains by measuring the state $H^{\otimes n}|0^n\rangle$ in the computational basis is the uniform distribution over $\{0,1\}^n$. In other words, prove that for all $y \in \{0,1\}^n$,

$$\Pr\left[\text{measure } H^{\otimes n}|0^n\rangle \text{ to be in state } |y\rangle\right] = 2^{-n}.$$

(This is an example of a quantum algorithm for generating random numbers, which you will explore in the next problem.)

Problem 2

Let S be a quantum system with Hilbert space $\mathcal{H}_S = \mathbb{C}^N$, and let $B = \{|b_0\rangle, |b_1\rangle, \ldots, |b_{N-1}\rangle\}$ be an orthonormal basis of $\mathcal{H}_S$.

(a) Prove that the following statement is *false*: there exists $U \in \mathrm{U}(N^2)$ such that for all $|\psi\rangle, |\phi\rangle \in \mathbb{C}^N$,

$$U|\psi\rangle|\phi\rangle = |\psi\rangle|\psi\rangle.$$

This is called the *no-cloning theorem*.

(b) Prove that for all $|\psi\rangle, |\phi\rangle \in \mathbb{C}^N$, there exists $U \in \mathrm{U}(N^2)$ such that

$$U|\psi\rangle|\phi\rangle = |\psi\rangle|\psi\rangle.$$

(*Hint:* Pay very close attention to the order of the quantifiers.)

(c) Explain why (b) does not violate the no-cloning theorem.

(d) Argue that (b) and the no-cloning theorem imply that experimental physicists can create many copies of a *known* quantum state but not an *arbitrary* quantum state.

Problem 3

In this problem, you will make an n-bit quantum random number generator in IBM's Qiskit. The following is a step-by-step guide to getting Qiskit up and running. By part (v), you should have everything you need to build your random number generator.

(a) Install Qiskit on your machine by following IBM's Qiskit Installation Guide at:[1]

https://docs.quantum.ibm.com/guides/install-qiskit

Note, it is recommended that you use Python virtual environments so that Qiskit is separated from other applications. Also, we will not be running jobs on actual quantum hardware, so you do *not* need to install `qiskit-ibm-runtime` in Step 3. Step 4 is also optional.

(b) In your virtual environment, install `qiskit-aer` by running

```
pip install qiskit-aer
```

This installs a vast suite of quantum computer simulators known as Qiskit Aer.

(c) With Qiskit installed, open a new file in your favorite text editor and name it "YOURTEAM-SNAME_1.py".

(d) As mentioned, the goal of this problem is to make a quantum circuit that acts as an n-bit random number generator. I will help you get started by implementing a different circuit.

In your .py file, insert:

```
from qiskit import *
qr = QuantumRegister(2,'q')
cr = ClassicalRegister(2, 'c')
```

The first line means "import everything from qiskit". In the second line, `qr` is a *quantum register*, which is just a collection of qubits whose overall state has not yet been specified. The command `QuantumRegister(2,'q')` tells Qiskit to construct a 2 qubit register, and to name the register '`q`'. Similarly, `cr` is a classical register, which is just a collection of bits whose values are not yet specified. The command `ClassicalRegister(2,'c')` tells Qiskit to construct a 2-bit register, and to name the register '`c`'. We need a classical register because that is where the measurement outcomes of the qubits will be stored.

To see what this all looks like, insert the following two lines and then run the file:[2]

```
qc = QuantumCircuit(qr,cr)
print(qc)
```

This makes a (rather trivial) *quantum circuit* with quantum register `qr` and classical register `cr`. As output, you should get something that looks like:

```
q_0:
q_1:
c: 2/
```

[1] If you ever need to know anything at all about Qiskit, it is documented here.
[2] There are nicer ways to view the circuit besides `print`. If you're interested, look here.

Here, `q_0` and `q_1` constitute the quantum register `qr`, and together represent individual qubits that are initialized in the state $|0\rangle$. (When we call the `QuantumCircuit` class, the qubits in the quantum register are all initialized to $|0\rangle$.) Also, the `2/` in the classical register indicates that it holds 2 bits, so in particular we can write 2 bits worth of information from the quantum register to the classical register by measuring the quantum register.

To prove to you that the state of each qubit is $|0\rangle$, and hence that the state of the two qubits is $|0\rangle \otimes |0\rangle$, insert the following:

```
from qiskit.quantum_info import Statevector
state = Statevector(qc)
print(state.data)
```

The first line is necessary because subpackages of Qiskit need to be imported separately. The next line calls `Statevector(qc)`, which asks Qiskit to construct the quantum state at the end of the circuit `qc`. The `data` attribute of `state` is the quantum state. Printing, you should get the following as output:

```
[1.+0.j 0.+0.j 0.+0.j 0.+0.j]
```

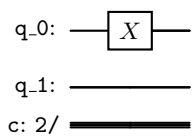
Indeed, up to transposition, this is $|0\rangle \otimes |0\rangle$ written in the computational basis (recall that in Python a number like `1.` denotes a floating point number and `j` denotes $\sqrt{-1}$). Now, let's make our circuit do something more than just the 4×4 identity gate I_4 (is it clear to you why `qc` is implementing I_4?). First, suppose we want our quantum circuit `qc` to put the first qubit in the state $|1\rangle$. To do this, we need to apply $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ to `q_0`. Indeed, you can check that $X|0\rangle = |1\rangle$. At the end of the day, we then expect the overall state of the two qubits to be the $\mathbb{C}^4$ computational basis state

$$\begin{aligned}(X \otimes I_2)|0\rangle \otimes |0\rangle &= |1\rangle \otimes |0\rangle \\ &= \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}.\end{aligned}$$

To do this in Qiskit, first remove or comment out the `state` from before, and replace it with

```
qc.x(0)
state = Statevector(qc)
```

Here, the first line says, "to `qc`, append an X gate to qubit `q_0`". In general, `qc.x(i)` applies an X gate to qubit `q_i`. If you now run `print(qc)`, you should see that indeed we are applying an X gate to qubit `q_0`:



Therefore, this sure seems like what we want to be doing. Let's see if it is by running `print(state.data)`. You should get

```
[0.+0.j 1.+0.j 0.+0.j 0.+0.j]
```

This is *almost* right, except that it corresponds to the computational basis state $|0\rangle\otimes|1\rangle$ and not $|1\rangle \otimes |0\rangle$, which is what we want! Alas, we have stumbled upon an annoying convention of Qiskit. Contrary to how we (and many others, like Nielsen and Chuang) talk about the order of qubits in a circuit—namely, by placing the qubit on the top line of the quantum circuit (`q_0`) in the left-most position in the ket, the qubit on the second-to-top line of the circuit (`q_1`) to the second-left-most position in the ket, and so on, so that the state for `qc` is $|$`q_0q_1`$\rangle$—Qiskit uses the reverse convention, and places the qubit on the top line (`q_0`) in the *right*-most position of the ket, the qubit on the second-to-top line (`q_1`) to the second-*right*-most position in the ket, and so on, so that to Qiskit the state of `qc` is actually $|$`q_1q_0`$\rangle$. To learn more about this, click here (it is related to the convention of whether in a bit string $q_0 q_1 \dots q_n$ you regard q_0 as the least or most significant bit).

To force Qiskit to use our convention (in which q_0 is the most significant bit), we can force `qc` to use a temporary "reverse ordering convention", and then compute the state vector:

```
state = Statevector(qc.reverse_bits())
print(state.data)
```

Here, the output is

```
[0.+0.j 0.+0.j 1.+0.j 0.+0.j]
```

which is the matrix form of $|1\rangle \otimes |0\rangle$, according to our convention. Take note that `reverse_bits()` literally swaps `q_0` and `q_1` in the quantum circuit `qc`, so if you try to do something sneaky and permanently *redefine* `qc` as `qc = qc.reverse_bits()`, then the circuit is going to look reversed when you print it. I discourage such permanent redefining because this reversal can quickly get confusing.

Now, let us make a slightly more complicated circuit. In addition to putting the first qubit `q_0` into state $|1\rangle$, let us now put the second qubit `q_1` into the superposition state $|+\rangle := \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. To do this, we need to apply the Hadamard gate H to `q_1`. Indeed, you can check that

$$\begin{aligned}(X \otimes H)|0\rangle \otimes |0\rangle &= |1\rangle \otimes \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right) \\ &= \frac{1}{\sqrt{2}}\left(|1\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle\right) \\ &= \begin{pmatrix} 0 \\ 0 \\ \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}.\end{aligned}$$

To do this in Qiskit, right after the `qc.x(0)` command from above, put

```
qc.h(1)
```

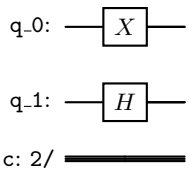
This says, "to `qc`, append an H gate to qubit `q_1`". Again running

```
state = Statevector(qc.reverse_bits())
print(state.data)
```

you should get

```
[0.+0.j 0.+0.j 0.70710678+0.j 0.70710678+0.j]
```

Since $\frac{1}{\sqrt{2}} \approx 0.707106$, this agrees with the state above.
If you want to be doubly sure that this circuit is indeed applying an X gate to qubit `q_0` and an H gate to qubit `q_1`, you can reprint the circuit by running `print(qc)`. It should look something like:



(If you had permanently redefined `qc` as `qc = qc.reverse_bits()`, which again I discourage, then you will not get the above result, but rather a reversed version.)
At this point, we have a quantum circuit which maps $|0\rangle \otimes |0\rangle$ to $|1\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Let us now see how to measure the output state, and write the measurement result to the classical register.
For a single computational basis measurement of both qubits, simply insert:

```
qc.measure(0,0)
qc.measure(1,1)
```

This tells Qiskit to measure qubit `q_0` in the computational basis, and record the result (which is either 0 or 1) into the first bit of the classical register, and then to measure qubit `q_1` in the computational basis, and record the result into the second bit of the classical register. In general, `qc.measure(i,j)` tells Qiskit to measure qubit `q_i` in the computational basis, and record the result into the `(j + 1)`th bit of the classical register. You should print the quantum circuit again to see how adding a measurement changes the circuit.
Now is the time to actually *run* the circuit and get a 2-bit output. To do this, we will use one of the most basic simulators in Qiskit Aer, which is a vast suite of quantum circuit simulators. In particular, we will use the `QasmSimulator`. To use this simulator, import it using

```
from qiskit_aer import QasmSimulator
```

Finally, to run the circuit, say 100 times, write

```
backend = QasmSimulator()
result = backend.run(qc.reverse_bits(), shots=100).result()
counts = result.get_counts()
```

Here, `backend` specifies that we are using the Qiskit Aer `QasmSimulator` to simulate our quantum circuit, and `backend.run(qc.reverse_bits(), shots=100)` simulates our circuit `qc` 100 times, and uses our reverse ordering convention. Use `.result()` to get the data from the 100 trials, and additionally call `.get_counts()` on `result` to put the data into a dictionary. You should `print(count)` to understand what the output looks like.

You are now ready to build an n-bit quantum random number generator. You should comment out or delete all the above code, and start fresh for the final, unguided part of this problem.

(e) In Qiskit, build an n-bit quantum random number generator, i.e., a Python program that takes an input an integer n, and then outputs a quantum circuit with an n-qubit register (that writes into a classical n-bit register) that on input $|0^n\rangle = |0\rangle^{\otimes n}$ outputs $y \in \{0,1\}^n$ with probability 2^{-n} when measured in the computational basis. (*Hint: recall problem 1 above!*). For $n = 4$, use Matplotlib or some other Python graphing software and output a histogram of 10 000 trials of your $n = 4$ circuit to convince me and yourself that it is indeed a 4-bit random number generator. The x-axis should contain the 16 4-bit strings 0000, 0001, etc., and the y-axis should specify the number of times a given string is measured.

(f) Submit to Canvas both your .py file and a picture of your histogram.